

PROGRAMAÇÃO DE COMPUTADORES III

Seminário — Programação Orientada a Testes

TEMA 8 — MÉTRICAS E COBERTURA DE CÓDIGO (COVERAGE)

G08PCOM3.01 — T01 (2026.2)

Integrantes	Gabriel Martins Nunes Rafaela Garcia Bernardes
Tema	Métricas e Cobertura de Código (Coverage)
Ferramentas	Python, pytest e coverage.py
Duração	20 minutos, rigorosamente cronometrados
Formato	Fundamentação teórica + demonstração prática / live coding

Finalidade deste documento

Formalizar a divisão de responsabilidades, os objetivos, a metodologia, o roteiro temporal, a demonstração prática, a estrutura do código e os pontos de domínio técnico necessários para uma apresentação rigorosa e comparativa.

1. Identificação e delimitação do tema

O seminário será desenvolvido sobre Métricas e Cobertura de Código (Coverage), com foco no uso do `coverage.py` integrado ao `pytest`. A apresentação deverá explicar e demonstrar como medir a execução de uma suíte de testes, interpretar seus resultados e compreender as limitações da métrica.

A abordagem será experimental: primeiro será apresentado um conjunto de testes insuficiente; depois sua cobertura será medida; em seguida serão acrescentados casos para exercitar caminhos não cobertos; por fim será discutido por que 100% de cobertura não equivale à ausência de defeitos.

2. Problema central

A questão norteadora será:

“Se todos os testes disponíveis estão passando, como podemos saber se eles realmente exercitam partes relevantes do código?”

A resposta será construída pela distinção entre execução dos testes, cobertura do código e correção do comportamento. O `coverage.py` mede quais partes do código foram executadas durante os testes; ele não substitui a elaboração de testes corretos.

3. Objetivos

Objetivo geral: apresentar e demonstrar o conceito de Code Coverage utilizando `pytest` e `coverage.py`, analisando criticamente o significado e as limitações dos percentuais de cobertura.

Objetivos específicos:

- Definir Code Coverage e sua finalidade.
- Diferenciar statement coverage e branch coverage.
- Demonstrar testes com `pytest` sob medição do `coverage.py`.
- Interpretar relatórios e localizar código não exercitado.
- Adicionar testes para cobrir caminhos ausentes.
- Demonstrar a evolução da cobertura.
- Explicar por que 100% não garante ausência de bugs.
- Entregar código organizado e reproduzível.

4. Tese central

“Code Coverage é uma métrica de abrangência dos testes: uma cobertura maior indica que uma parcela maior do código foi exercitada, mas não permite concluir, isoladamente, que o software está correto ou livre de bugs.”

5. Divisão formal entre Gabriel e Rafaela

Gabriel — teoria: problema; conceito de Code Coverage; `coverage.py`; statement coverage; relação entre `pytest` e `coverage`.

Gabriel — prática: apresentar o código inicial; executar o primeiro teste; demonstrar teste passando com cobertura insuficiente; executar `coverage.py` e iniciar a análise.

Rafaela — teoria: branch coverage; interpretação dos relatórios; cobertura versus qualidade; limitações; 100% não significa ausência de bugs.

Rafaela — prática: adicionar testes faltantes; executar novamente; comparar resultados; gerar/abrir relatório HTML; conduzir a demonstração da limitação dos 100%.

Regra de equilíbrio: ambos devem conhecer o projeto completo e estar aptos a responder perguntas sobre qualquer etapa.

6. Roteiro temporal — 20 minutos

Eta pa	Conteúdo	Responsável	Tempo
1	Abertura e objetivo	Ambos	0:30
2	Problema e conceito	Gabriel	1:00
3	coverage.py e statement coverage	Gabriel	1:30
4	Branch coverage	Rafaela	1:30
5	Arquitetura da demonstração	Rafaela	1:00
6	Código inicial + teste	Gabriel	2:00
7	Coverage inicial + análise	Gabriel	2:30
8	Novos testes + execução	Rafaela	2:30
9	Relatório final + comparação	Rafaela	2:00
10	100% ≠ ausência de bugs	Ambos	3:00
11	Conclusão	Ambos	1:30

Ensaiai para terminar entre 18:30 e 19:30, mantendo margem de segurança antes do limite de 20 minutos.

7. Estrutura dos slides

#	Título	Conteúdo	Responsável
1	Métricas e Cobertura	Tema, disciplina e integrantes	Ambos
2	O problema	Testes passando ≠ cobertura suficiente	Gabriel
3	O que é Coverage?	Definição, finalidade e conceito	Gabriel
4	coverage.py	Medição e relatórios	Gabriel
5	Statement × Branch	Diferenças e caminhos	Rafaela
6	Nossa demonstração	Fluxo experimental	Rafaela
7	Primeiro cenário	Teste passa, cobertura incompleta	Gabriel
8	Segundo cenário	Novos testes e aumento	Rafaela
9	Leitura do relatório	Stmts, Miss, Branch, BrPart, Cover	Rafaela
10	100% não significa ausência de bugs	Limitações e contraexemplo	Ambos
11	Conclusão	Síntese e referências	Ambos

8. Metodologia prática

Será utilizado um módulo simples de cálculo de desconto. O domínio é propositalmente pequeno, mas contém múltiplos caminhos de decisão, permitindo demonstrar statement coverage e branch

coverage sem comprometer o limite de 20 minutos.

Estrutura do projeto

```
seminario_pc3_coverage/
├── apresentacao.pdf
├── README.md
├── requirements.txt
├── src/
│   ├── loja.py
│   ├── tests/
│   └── test_loja.py
```

Implementação inicial

```
def calcular_desconto(valor, cliente_vip):
    if valor < 0:
        raise ValueError("O valor não pode ser negativo.")

    if cliente_vip:
        return valor * 0.80

    if valor >= 500:
        return valor * 0.90

    return valor
```

Primeiro teste — propositalmente insuficiente

```
from src.loja import calcular_desconto

def test_cliente_vip():
    assert calcular_desconto(100, True) == 80
```

Executar primeiro pytest e mostrar que o teste passa. Em seguida:

```
coverage run --branch -m pytest
coverage report -m
```

O ponto pedagógico é evidenciar: o teste passa, mas existem linhas e/ou branches não exercitados.

9. Segunda etapa — ampliar a suíte

```
import pytest
from src.loja import calcular_desconto

@pytest.mark.parametrize(
    "valor,vip,esperado",
    [
        (100, True, 80),
        (100, False, 100),
        (500, False, 450),
        (1000, False, 900),
    ],
)
def test_calcular_desconto(valor, vip, esperado):
    assert calcular_desconto(valor, vip) == esperado

def test_valor_negativo():
    with pytest.raises(ValueError):
        calcular_desconto(-1, False)
```

Executar novamente:

```
coverage run --branch -m pytest
coverage report -m
coverage html
```

Os percentuais reais devem ser os produzidos pela execução. Não inserir números fictícios nos slides.

10. Interpretação do relatório

Campo	Significado
Stmts	Número de instruções executáveis consideradas.
Miss	Instruções que não foram executadas.
Branch	Possibilidades de desvio/caminhos identificados.
BrPart	Branches parcialmente cobertos.
Cover	Percentual de cobertura apresentado pelo relatório.

Abrir o relatório HTML durante a apresentação para visualizar as linhas não cobertas. A evidência visual é mais forte do que mostrar apenas o percentual.

11. Statement Coverage × Branch Coverage

```
if idade >= 18:  
    return "adulto"  
else:  
    return "menor"
```

Statement coverage: verifica se instruções foram executadas.

Branch coverage: verifica se os caminhos possíveis das decisões foram exercitados.

Um teste com idade = 20 percorre o caminho verdadeiro, mas não o caminho correspondente a idade < 18. Essa distinção deverá ser evidenciada no relatório.

12. Demonstração crítica — 100% não significa ausência de bugs

Mesmo que todos os elementos considerados pela métrica tenham sido executados, os testes podem estar verificando uma implementação incorreta, uma regra de negócio equivocada ou casos de entrada insuficientes.

100% de cobertura = todo o código considerado foi exercitado.

Não significa 100% de correção, ausência de bugs ou cobertura de todas as combinações relevantes de dados e requisitos.

13. Comandos da demonstração

Instalação:

```
pip install pytest coverage
```

Execução:

```
pytest
```

Medição com branches:

```
coverage run --branch -m pytest
```

Relatório textual:

```
coverage report -m
```

Relatório HTML:

O plugin `pytest-cov` pode ser citado como alternativa de integração, mas o foco deverá permanecer no `coverage.py`.

14. Qualidade e reprodutibilidade

- Nomes claros e organização PEP 8.
- Separação entre código-fonte e testes.
- Testes independentes e determinísticos.
- `requirements.txt`.
- `README.md` com instalação e execução.
- Sem dependências externas desnecessárias.
- Projeto testado previamente em ambiente limpo.

15. Checklist de ensaio

- ☐ Executar o projeto do zero.
- ☐ Confirmar descoberta dos testes pelo `pytest`.
- ☐ Confirmar teste passando com cobertura incompleta.
- ☐ Confirmar relatório HTML.
- ☐ Confirmar cenário final após novos testes.
- ☐ Confirmar `--branch`.
- ☐ Verificar legibilidade no projetor.
- ☐ Cronometrar os dois participantes.
- ☐ Ensaiar a explicação de 100%.
- ☐ Ensaiar perguntas técnicas.
- ☐ Levar cópia local dos arquivos.
- ☐ Gerar ZIP final conforme SIGAA.

16. Perguntas que ambos devem dominar

100% de coverage significa que não existem bugs?

Não. Significa que, segundo o critério utilizado, os elementos considerados foram exercitados. A correção depende da qualidade dos testes.

Coverage substitui testes?

Não. Coverage mede execução; não cria nem garante a qualidade das asserções.

Qual a diferença entre `pytest` e `coverage.py`?

`pytest` executa e avalia os testes; `coverage.py` mede quais partes do código foram executadas durante essa execução.

O que é `branch coverage`?

Medição dos diferentes caminhos de decisão, como verdadeiro e falso de uma condição.

Por que `branch coverage` é útil?

Porque uma linha pode ser executada sem que todos os caminhos de uma decisão sejam exercitados.

Coverage alto garante software de qualidade?

Não. É uma evidência de abrangência, que deve ser combinada com testes bem projetados e outras práticas de qualidade.

17. Estrutura final para o SIGAA

```
seminario_pc3_coverage.zip
├── apresentacao.pdf
├── README.md
├── requirements.txt
├── src/
│   ├── loja.py
│   └── tests/
│       └── test_loja.py
```

Os dois nomes devem constar claramente nos arquivos. A apresentação e o código devem estar dentro de um único arquivo compactado, conforme o enunciado.

18. Conclusão prevista

A conclusão deverá retomar a pergunta inicial: testes passando não significam, por si só, que todo o código relevante foi exercitado. O coverage.py torna essa abrangência mensurável e permite localizar lacunas na suíte. Branch coverage amplia a análise ao considerar caminhos de decisão.

Mensagem final: “Coverage ajuda a responder quanto do código foi exercitado; a qualidade dos testes determina se aquilo que foi exercitado foi realmente verificado de maneira significativa.”

19. Referências técnicas

- Coverage.py — documentação oficial: <https://coverage.readthedocs.io/>
- Coverage.py — Branch Coverage: <https://coverage.readthedocs.io/en/latest/branch.html>
- pytest — documentação oficial: <https://docs.pytest.org/>
- pytest — parametrização: <https://docs.pytest.org/en/stable/how-to/parametrize.html>
- pytest-cov — documentação oficial: <https://pytest-cov.readthedocs.io/>

Observação: os percentuais de cobertura exibidos na apresentação deverão ser os valores efetivamente produzidos pela execução do projeto.